

PATERNI PONASANJA

1. Strategy - Predlozak entitet ima atribut Pretplata (true/false), pa se isti modul ponašava drugačije u zavisnosti od tipa predloška. Takođe posjeduje atribut Period. Strategy patern omogućava da se različita logika obrade entiteta izdvoji u zasebne klase umjesto ponavljanja if/else grananja unutar kontrolera za svaku razlicitu vrijednost perioda, te na osnovu da li je u pitanju pretplata ili ne.
2. State - Klasa Racun ima atribut Aktiviran (Boolean) koji označava da li je račun aktiviran ili neaktiviran. Sistem treba da razlikuje ta stanja racuna i na osnovu njih radi odredjene operacije, te koristi ASP.NET Identity sa ulogama Klijent, Uposlenik i Admin, gdje se pristup funkcionalnostima razlikuje u zavisnosti od uloge korisnika. State patern omogućava da se ponašanje računa i korisničkih prava definiše kroz stanja umjesto kroz uslovne provjere u kontrolerima.
3. Template Method - Kontroleri TransakcijaController, RacunController, OsobaController implementiraju slične get i set metode. Template Method patern omogućava da se zajednička logika definiše u baznoj klasi, dok izvedene klase implementiraju samo specifične dijelove za svaki entitet.
4. Chain of Responsibility - Prije izvršenja transakcije potrebno je proći niz provjera: da li je račun aktivan, da li ima dovoljno sredstava, da li je kursna lista ažurna. Bez ovog patern, sve provjere bi bile grupisane u jednoj metodi, što otežava dodavanje novih pravila validacije. Chain of Responsibility omogućava da svaka provjera bude zaseban handler koji proslijeđuje zahtjev sljedećem u lancu.
5. Command - Entitet Predlozak sa Pretplatama i Period (Dnevno, Sedmicno, Mjesecno, Godisnje) zahtijeva automatizovano i ponavljajuće izvršavanje plaćanja. Operacije poput otkazivanja transakcije predstavljaju složene akcije koje zahtijevaju poništavanje prethodnih promjena. Command patern enkapsulira svaku operaciju kao zaseban objekat, što olakšava automatizaciju pretplata, vođenje historije i poništavanje operacija.

6. Iterator - Sistem mora omogućiti pregled liste transakcija, računa, predložaka i kursnih lista bez otkrivanja interne strukture podataka. Bez Iterator paterna, svaki modul bi morao poznavati način pohrane i iteracije kroz kolekcije entiteta. Iterator omogućava jednostavan način prolaska kroz kolekcije, nezavisno od toga da li su podaci u bazi, listi ili drugoj strukturi.

7. Mediator - Transakcija zahtijeva koordinaciju između više entiteta: Osoba, Racun, Kurs i Predlozak, bez da svaka klasa direktno komunicira sa svim ostalima. Bez Mediator paterna, entiteti bi bili međusobno čvrsto povezani, što otežava održavanje i testiranje. Mediator centralizuje komunikaciju između objekata, smanjujući direktne zavisnosti između klasa.

8. Observer - Nakon izvršenja transakcije ili promjene stanja računa, korisnici različitih uloga (Klijent, Uposlenik, Admin) trebaju biti obaviješteni. Bez Observer paterna, modul za transakcije mora direktno pozivati servise za email i ažuriranje stanja računa, što stvara čvrste veze između klasa. Observer omogućava da se novi kanali obavještenja dodaju bez izmjene logike transakcije.

9. Visitor - Projekat sadrži zasebne Index i Details view-ove za pojedine entitete (Racun, Transakcija, Predlozak, Kurs), pri čemu svaki prikazuje drugačija polja. Visitor patern omogućava dodavanje novih operacija nad postojećim entitetima (npr. prikaz u Admin dashboardu) bez mijenjanja klasa modela.

10. Interpreter - Predlozak sadrži podatke o plaćanju (iznos, period, svrha uplate) koji se moraju interpretirati i primijeniti pri automatskom izvršavanju transakcije. Pravila za periodična plaćanja (Dnevno, Sedmicno, Mjesecno, Godisnje) sistem mora razumjeti i izvršiti. Interpreter patern omogućava definisanje i interpretaciju ovih pravila bez hardkodiranja logike u kontrolerima.

11. Memento - Pri stopiranju transakcije ili greške u obradi, sistem mora vratiti prethodno stanje računa i transakcije. Bez Memento paterna, vraćanje stanja zahtijeva ručno čuvanje i restauraciju podataka u više mjesta koda. Memento omogućava snimanje trenutnog stanja objekta i kasnije vraćanje na to stanje bez otkrivanja interne implementacije.